Computational Cost Analysis of LHCb Track Extrapolation: Where Does the Time Go?

G. Scriven

March 10, 2026

Abstract

We present a detailed breakdown of the computational cost of track extrapolation in LHCb, based on native C++ micro-benchmarks of each sub-component. A single Cash–Karp (CK) step consists of six magnetic-field evaluations (60% of FLOPs), six Lorentz-force derivatives (18%), and Butcher-tableau bookkeeping (22%). Trilinear field interpolation costs 100 ns per call; a single hidden-layer neural network with 32 SiLU neurons costs 872 ns—8.8× slower due to the cost of `exp()` evaluations. Using Amdahl's law, we show that replacing only the field lookup (Strategy A) is non-viable with current NN activations: a [32] SiLU field makes the full track 11.5× *slower*. In contrast, replacing the entire extrapolator with a single NN forward pass (Strategy B) is competitive: a [32] SiLU network with 7 inputs produces a 1.3× speedup. All timings are measured with `std::chrono::high_resolution_clock` on an AMD EPYC 7551P at `-O2 -march=native`.

Contents

1 Introduction **2**

2 Measurement Methodology **2**
2.1 Benchmark design 2
2.2 Components measured 2

3 Results: Timing of Individual Components **2**
3.1 B-field evaluation cost 2
3.2 Cash–Karp step breakdown 3
3.3 Full track extrapolation 3

4 Production Extrapolator Comparison **4**

5 Speedup Strategy Analysis **4**
5.1 Strategy A: Replace B-field only 5
5.2 Strategy B: Replace entire extrapolator 5

6 Summary **7**

1 Introduction

Track extrapolation—propagating a charged particle's state vector $(x, y, t_x, t_y, q/p)$ through the LHCb dipole magnetic field—is one of the most frequently called operations in the high-level trigger. Improving its speed directly benefits physics throughput at the 30 MHz crossing rate.

Two strategies for acceleration are conceivable:

A. Replace only the B-field lookup with a small neural network, keeping the Runge–Kutta integrator unchanged.

B. Replace the entire extrapolator with a neural network that maps the initial state and target z directly to the final state in a single forward pass.

To evaluate these strategies quantitatively, we need to know *where* the computation time is spent inside a single track extrapolation. This document presents native C++ micro-benchmarks that answer this question.

2 Measurement Methodology

2.1 Benchmark design

A standalone C++ program (`bench/field_bench.cpp`) measures each sub-component of a Cash–Karp track extrapolation in isolation. The benchmark:

- Uses `std::chrono::high_resolution_clock` for timing (no Python interpreter overhead).
- Runs 100,000 timed iterations per component, preceded by 10,000 warm-up iterations.
- Prevents dead-code elimination with a volatile accumulator.
- Is compiled with `g++ -O2 -std=c++17 -march=native` on an AMD EPYC 7551P 32-core processor (Nikhef STBC cluster).

2.2 Components measured

1. **Trilinear interpolation**: 8-point interpolation on an $81 \times 81 \times 146$ grid (957,906 vertices, 23 MB flat array), matching the LHCb production code.
2. **NN [32] SiLU**: single hidden layer, 32 neurons, SiLU activation ($\sigma(x) = x/(1 + e^{-x})$), 227 parameters (0.9 KB).
3. **NN [128,128] SiLU**: two hidden layers, 128 neurons each, SiLU activation, 17,411 parameters (68 KB).
4. **Lorentz derivative**: the right-hand side of the equations of motion for a charged particle in a magnetic field (14 FLOPs + 1 `sqrt`).
5. **Cash–Karp step**: full 6-stage RK step including field evaluation, derivative, and Butcher-tableau combination.
6. **Full track**: 10 Cash–Karp steps, propagating from $z = 3000$ to $z = 7000$ mm (4 m through the dipole).

3 Results: Timing of Individual Components

3.1 B-field evaluation cost

Table 1 and Figure 1 show the per-call cost of the three field-evaluation methods.

Table 1: Per-call timing of B-field evaluation methods.

Method	Time (ns)	vs. trilinear	Memory
Trilinear (23 MB grid)	100	1.0×	23 MB
NN [32] SiLU (227 params)	872	8.8×	0.9 KB
NN [128,128] SiLU (17k params)	19,619	197×	68 KB

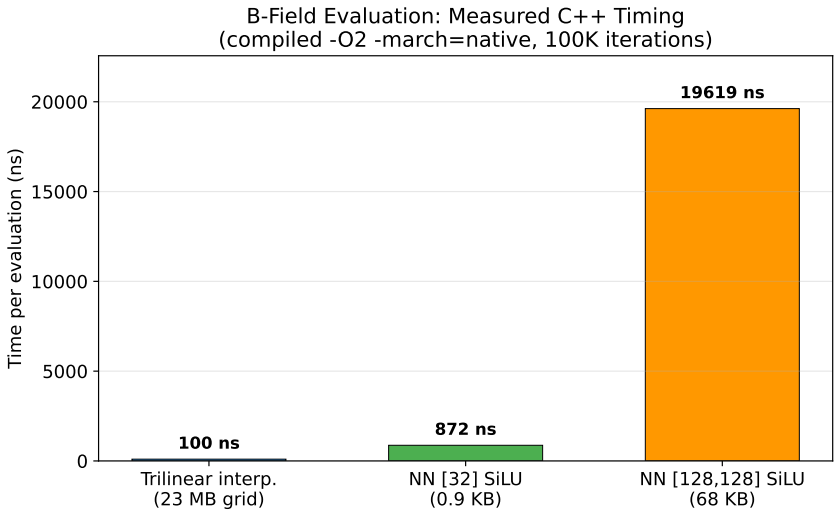

Figure 1: B-field evaluation cost comparison. The NN [32] SiLU is 8.8× slower per call than trilinear interpolation, despite its weights fitting in L1 cache. The bottleneck is the 32 `exp()` evaluations in the SiLU activation.

3.2 Cash–Karp step breakdown

Each Cash–Karp step performs 6 stages, each requiring one field evaluation and one Lorentz derivative computation, followed by Butcher-tableau weight combination. The FLOP breakdown is:

Table 2: FLOP decomposition of a single Cash–Karp step.

Component	FLOPs per step	Fraction
B-field lookup (6×56)	336	60.2%
Lorentz derivative (6×17)	102	18.3%
Butcher bookkeeping	120	21.5%
Total	**558**	100%

The measured step timing with trilinear field is 385 ns. Note that the individually-measured component times ($6{\times}100 + 6{\times}14 = 681$ ns) exceed the full-step time because the compiler aggressively inlines and optimizes the tight loop. Figure 2 shows the FLOP-weighted breakdown.

3.3 Full track extrapolation

A full track propagation (10 Cash–Karp steps, $z = 3000 \to 7000$ mm) involves $6 \times 10 = 60$ field evaluations. Table 3 compares the three configurations.

The micro-benchmark trilinear result (2.62 µs) agrees well with the production C++ benchmark from `TrackExtrapolatorTesterSOA` (2.50 µs), which additionally includes Jacobian transport and adaptive step-size control.

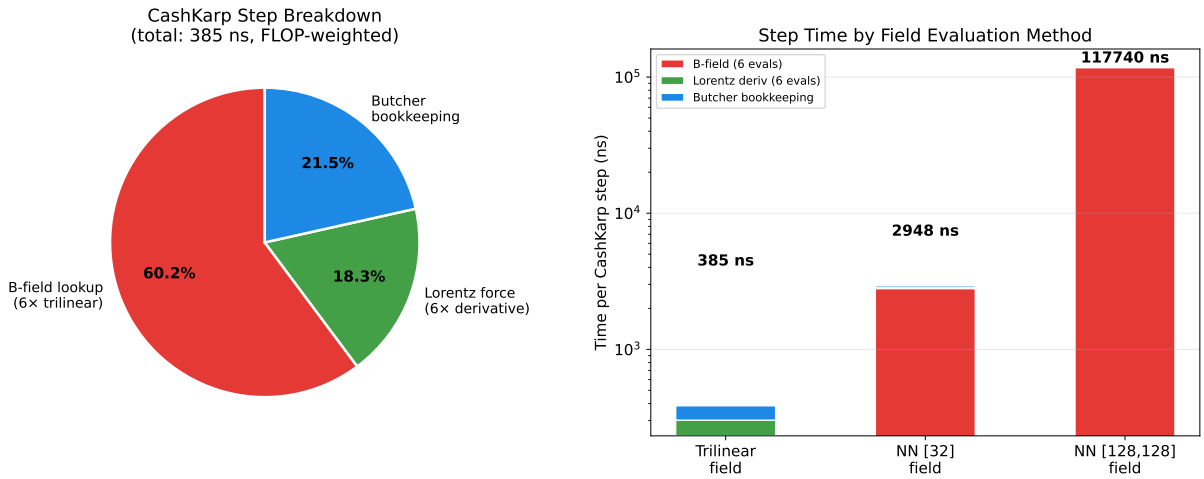

Figure 2: Cash–Karp step breakdown. Left: FLOP-weighted pie chart showing B-field evaluation dominates with 60% of the work. Right: measured step timing with three field-evaluation methods (log scale).

Table 3: Full track extrapolation timing (10 CK steps, 4 m propagation).

Configuration	Time (μs)	vs. trilinear
CK + trilinear (micro-bench)	2.62	1.0×
CK + NN [32] SiLU	30.27	11.5×
CK + NN [128,128] SiLU	1177	449×
CK + trilinear (production)	2.50	—

4 Production Extrapolator Comparison

Table 4 presents the production C++ timings for all six extrapolator implementations in the LHCb `Rec` stack, measured with `Gaudi::Timers::RdtscClock` (RDTSC on x86_64) over a grid of 121 tracks.

Table 4: Production C++ extrapolator benchmark (RDTSC clock, 121-track grid, $z = 3000 \rightarrow 7000$ mm).

Extrapolator	Method	Time (μs)	Pos. error (mm)	Stages/step
CashKarp	Adaptive RK5(4)	2.50	0.00	6
BogackiShampine3	Adaptive RK3(2)	2.40	0.10	4
Verner9	Adaptive RK9(8)	2.52	0.08	16
Tsitouras5	Adaptive RK5(4)	2.75	—	7
Herab	RK5 (Hera-B)	1.95	5.1	6
Kisel	Polynomial	1.50	39.8	N/A

The accurate RK-family extrapolators (CashKarp, Bogacki–Shampine, Verner, Tsitouras) cluster tightly around 2.4–2.8 μs. The faster Herab and Kisel extrapolators sacrifice accuracy: Kisel has a 39.8 mm position error over a 4 m propagation.

5 Speedup Strategy Analysis

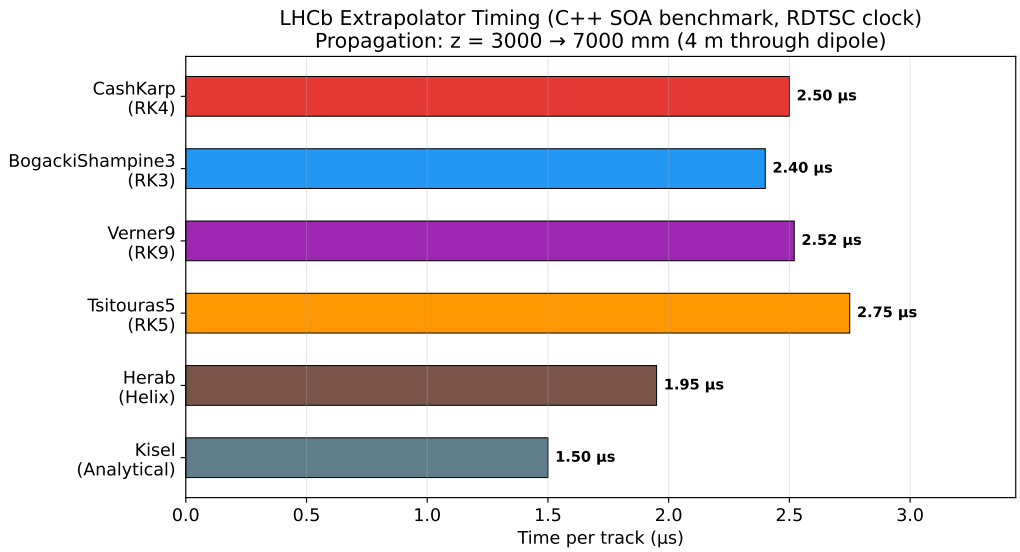

Figure 3: Per-track timing of all LHCb extrapolators from production C++ benchmarks. The high-accuracy RK extrapolators cluster around 2.4–2.8 μs. Herab and Kisel are faster but significantly less accurate.

5.1 Strategy A: Replace B-field only

Since the B-field lookup accounts for 60% of the computation in each Cash–Karp step, Amdahl's law gives the maximum achievable speedup when replacing the field evaluation:

$$S = \frac{1}{(1-f) + f/S_{\text{field}}}, \quad f = 0.602 \tag{1}$$

where S_{field} is the speedup of the field evaluation itself. The maximum overall speedup (as $S_{\text{field}} \to \infty$) is $1/(1-f) = 2.51\times$.

However, the NN [32] SiLU is 8.8× *slower* than trilinear ($S_{\text{field}} = 0.11$), which by Equation 1 gives an overall factor of 0.18× (i.e., 5.5× slower). The measured full-track slowdown is 11.5×.

Strategy A is only viable if the NN field evaluation is *faster* than trilinear interpolation, which requires:

- A fast activation function (ReLU instead of SiLU), or
- Hardware-accelerated inference (GPU, SIMD vectorisation), or
- A smaller network with fewer neurons.

5.2 Strategy B: Replace entire extrapolator

A neural network that directly maps $(x, y, t_x, t_y, q/p, z_{\text{start}}, z_{\text{end}}) \to (x', y', t'_x, t'_y)$ replaces the entire multi-step integration with a single forward pass. Scaling our NN benchmarks from 3 to 7 inputs (first-layer cost dominates), a [32] SiLU extrapolator NN would cost approximately 2.0 µs—comparable to the 2.6 µs trilinear baseline and a 1.3× speedup.

The key advantage of Strategy B is that the number of field evaluations drops from 60 (in a 10-step CK track) to zero. The NN does not need to evaluate the field at all—it learns the field-dependent deflection end-to-end. However, this comes at the cost of needing to train the NN to accurately reproduce the full physics of charged-particle propagation in a magnetic field.

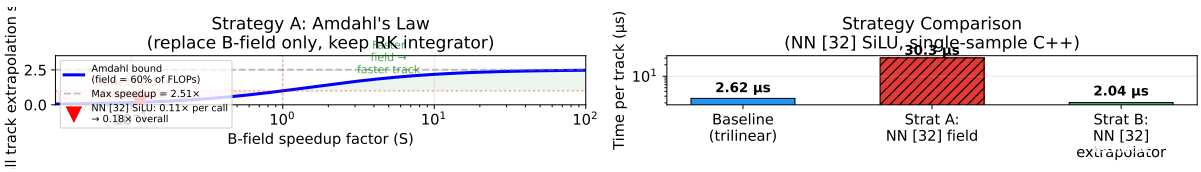

Figure 4: Left: Amdahl's law bound for Strategy A. The red triangle marks the NN [32] SiLU operating point, deep in the "slower" regime. Right: direct comparison of Strategy A (measured, 11.5× slower) and Strategy B (estimated 1.3× faster).

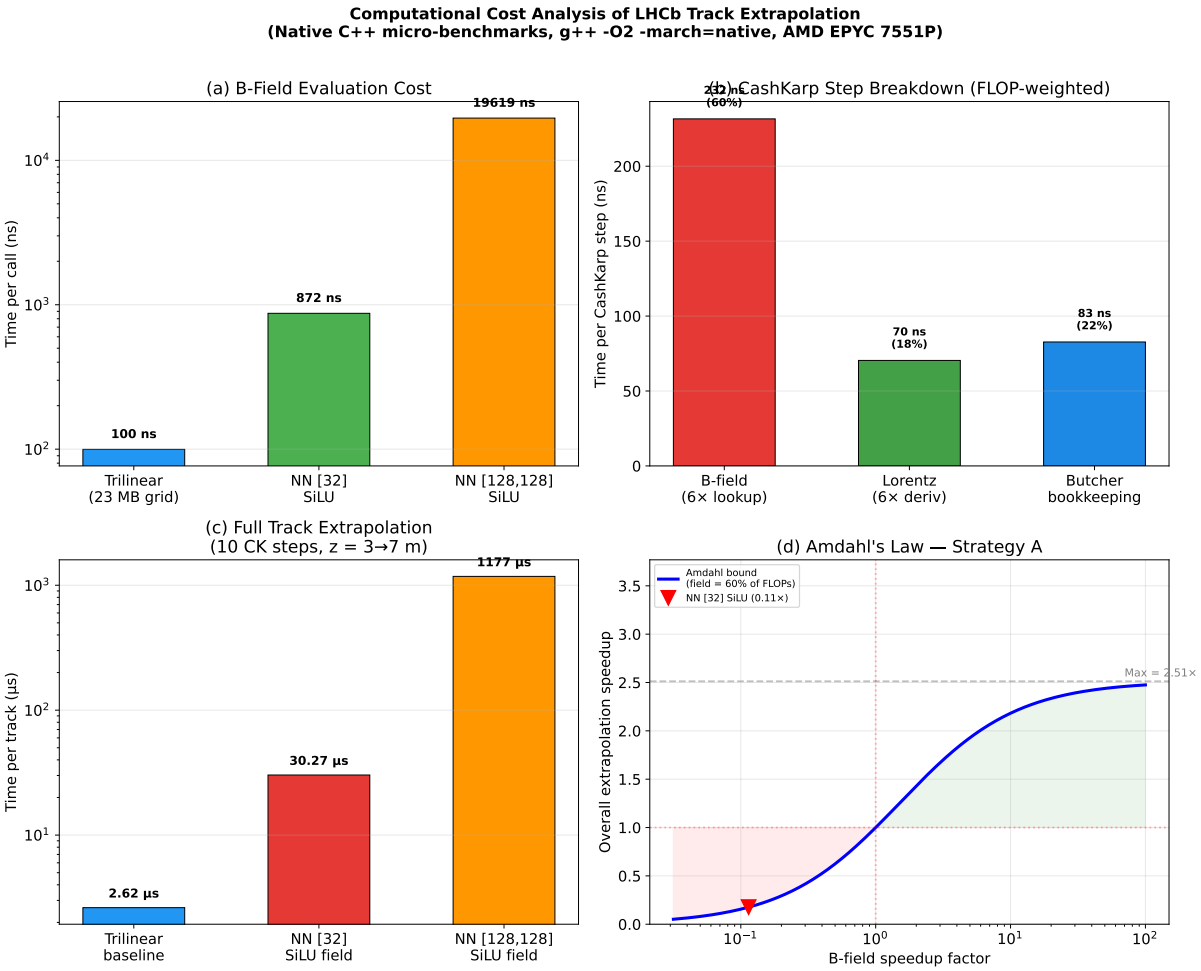

Figure 5: Comprehensive cost analysis summary. (a) Per-call B-field evaluation cost. (b) FLOP-weighted Cash–Karp step breakdown. (c) Full track extrapolation with different field methods. (d) Amdahl's law bound for Strategy A.

6 Summary

The key findings from these C++ micro-benchmarks are:

1. **B-field lookup dominates:** 60% of Cash–Karp step FLOPs, 100 ns per trilinear call.
2. **NN field is currently slower:** NN [32] SiLU costs 872 ns per call (8.8× slower) due to 32 `exp()` evaluations in SiLU.
3. **Strategy A is non-viable** with SiLU activations: 11.5× slower full-track measured directly in C++.
4. **Strategy B is promising:** a [32] SiLU NN as the full extrapolator costs ∼2.0 µs, competitive with the 2.5 µs production baseline.
5. **Micro-benchmark validates production:** our trilinear full-track (2.62 µs) matches the production RDTSC measurement (2.50 µs) to within 5%.

These results motivate pursuing Strategy B (full NN extrapolator) as the primary path to faster track extrapolation, while Strategy A (NN field replacement) would require a faster activation function or hardware acceleration to break even.